

--	--	--

Mercurial Music Recommendation App

Project Report

Submitted by

Prajwal Bagchi & Ranveer Chaudhary

(2110993821) & (2110993886)



BE-CSE (ARTIFICIAL INTELLIGENCE)

Guided by:

Dr. Kamal Deep Garg

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING &
TECHNOLOGY**

CHITKARA UNIVERSITY, RAJPURA

		1
--	--	---

--	--	--

TABLE OF CONTENT

1. Abstract
2. Introduction
3. Background
 - 3.1 Machine Learning
 - 3.2 K-Means Clustering
 - 3.3 K-Nearest Neighbour
 - 3.4 K-Means++ Clustering
4. Dataset
5. Implementation
 - 5.1 Data Collection
 - 5.2 Data Preprocessing and Data Visualization
 - 5.3 Feature Engineering
 - 5.4 Model Architecture
 - 5.5 Model Training
 - 5.6 Evaluation Metrics
 - 5.7 Results
6. Flowchart
7. Experimental Results
8. Result Summary
 - 8.1 Result information gained from models used
9. Conclusion

		1
--	--	---

--	--	--

ACKNOWLEDGEMENT

With immense pleasure we, Mr. Prajwal Bagchi and Mr. Ranveer Chaudhary presenting “MERCURIAL: A Music Recommendation Application” project report as part of the curriculum of ‘BE-CSE(AI)’.

We would like to express our sincere thanks to Dr. Kamal Deep Garg, for his valuable guidance and support in completing our project.

We would also like to express our gratitude toward our dean Dr. Sushil Kumar Narang for giving us this great opportunity to do a project on Music Recommendation . Without their support and suggestions, this project would not have been completed.

--	--	--

1. Abstract

This report presents the development and evaluation of a music recommendation application that utilizes the K-means++ clustering algorithm in conjunction with the Flask framework. The objective of the application is to provide personalized music recommendations to users based on their preferences and listening habits. The K-means++ clustering algorithm is employed to create optimal clusters of songs based on their audio features, enabling the identification of similarities among songs and users.

Prominent music streaming platforms such as Spotify and Pandora utilize K-means++ clustering to group songs with similar characteristics into clusters. By leveraging this clustering technique, these platforms are able to offer personalized song recommendations to their users, leading to increased user engagement and satisfaction.

In our evaluation, the developed music recommendation application showcased impressive performance, accurately recommending songs that align with users' musical preferences. The application's integration with the Flask framework ensures seamless and efficient handling of user requests, providing a smooth user experience.

The significance of a robust music recommender cannot be overstated. By offering tailored music recommendations, these applications not only enhance user satisfaction but also facilitate music discovery, broadening users' musical horizons. This, in turn, drives engagement and retention for music streaming platforms, leading to increased user loyalty and business growth.

The findings of this project highlight the value of leveraging clustering algorithms, such as K-means++, in developing efficient and accurate music recommendation systems. The real-life examples of successful applications employing K-means++ clustering underscore its practicality and effectiveness in the field of music recommendation.

This report contributes to the existing body of knowledge by showcasing the practical application of K-means++ clustering in music recommendation and emphasizing the need for continued research and innovation in this domain. The developed application serves as a testament to the potential of K-means++ clustering in enhancing the music listening experience for users and sets the stage for further advancements in the field.

		1
--	--	---

--	--	--

2. Introduction

In the digital age, the availability of vast music libraries and streaming platforms has made music discovery an exciting yet challenging endeavor for users. To address this challenge, music recommendation systems have emerged as indispensable tools for delivering personalized music suggestions to users. In this project, we present the development and evaluation of MERCURIAL, an innovative music recommendation application that harnesses the power of machine learning and clustering techniques,

The primary goal of MERCURIAL is to leverage the K-means++ clustering algorithm to group songs based on their audio features, enabling the identification of underlying patterns and similarities. K-means++ clustering, a popular unsupervised learning algorithm, partitions a dataset into optimal clusters based on specified parameters. In the context of MERCURIAL, key parameters such as the number of clusters and convergence criteria are carefully selected to ensure the clustering results accurately represent the inherent structure of the music dataset.

The deployment of MERCURIAL on HTML, CSS, JavaScript, and Python Flask framework offers several advantages. HTML provides the structure and layout for the web interface, while CSS enables the customization and styling of the application. JavaScript enhances the interactivity by implementing dynamic elements and user interactions. Python Flask, a lightweight web framework, facilitates the integration of the machine learning model and handles user requests efficiently, allowing for real-time music recommendations.

By incorporating machine learning and clustering techniques, MERCURIAL goes beyond simple rule-based approaches and offers personalized music recommendations that cater to individual preferences and listening habits. The application considers various factors, including genres, artist similarity, audio features (such as tempo, energy, and acousticness), and user feedback to generate highly accurate recommendations. Through an intuitive user interface, users can easily explore a diverse range of songs and artists, expanding their musical horizons.

The effectiveness of MERCURIAL's music recommendations will be evaluated through rigorous experiments and analyses. By measuring parameters such as precision, recall, and user satisfaction, we aim to demonstrate the application's ability to provide relevant and engaging recommendations. Additionally, the scalability and efficiency of the Flask deployment will be assessed to ensure smooth handling of user requests, even as the application user base grows.

		1
--	--	---

--	--	--

The results of this project will contribute to the field of music recommendation systems by showcasing the potential of K-means++ clustering and the seamless integration of HTML, CSS, JavaScript, and Python Flask framework for deployment. The unique combination of machine learning algorithms, clustering settings, and web technologies enables MERCURIAL to deliver personalized music recommendations, enriching users' music listening experiences.

3. Background

3.1 MACHINE LEARNING

Machine learning refers to a branch of artificial intelligence (AI) that focuses on developing algorithms and models capable of learning from data and making predictions or decisions without explicit programming. Machine learning algorithms analyze large datasets, identify patterns, and make predictions or classifications based on the information within the data. These algorithms are trained using historical data and then evaluated on new, unseen data to assess their performance.

Machine learning techniques can be applied in various ways:

1. **Data Classification:** Machine learning algorithms can assign data points to predefined categories or classes. For example, in the context of mobile phone pricing, a machine learning model can be trained to classify phones into different price ranges based on their specifications.
2. **Outcome Prediction:** Machine learning models can generate predictions about future events or values. For instance, in a healthcare study, a machine learning algorithm can be developed to predict the likelihood of a patient developing a certain disease based on their medical history and other relevant factors.
3. **Pattern Discovery:** Machine learning can uncover hidden patterns or relationships within data that may not be immediately apparent. This provides valuable insights and aids in making data-driven decisions. For example, in a marketing research paper, machine learning algorithms can be applied to identify patterns in customer behavior and preferences, helping companies tailor their marketing strategies.

--	--	--

4. Process Optimization: Machine learning algorithms can optimize complex processes by learning from data and improving performance over time. This may involve tasks such as resource allocation, scheduling, or anomaly detection.

It is important to consider that the selection of machine learning algorithms, preprocessing techniques, feature selection methods, and evaluation metrics will depend on the specific research problem and dataset characteristics. The research paper should provide a detailed explanation of the chosen machine learning approach, justify its suitability for the problem at hand, and present the experimental results and analysis to support the research findings.

3.2 K-Means Clustering

K-means clustering is an unsupervised machine learning algorithm commonly used in research papers for data clustering tasks. It aims to partition a given dataset into K distinct clusters, where each data point belongs to the cluster with the nearest mean (centroid).

The K-means algorithm follows these steps:

1. Initialization: Choose the number of clusters K. Randomly initialize K centroids.

2. Assignment: Assign each data point to the nearest centroid based on the Euclidean distance.

- Let $X = \{x_1, x_2, \dots, x_n\}$ be the dataset of n data points, where each data point x_i is a d-dimensional vector.

- Let $C = \{c_1, c_2, \dots, c_K\}$ be the set of K centroids.

- The assignment step calculates the Euclidean distance between each data point x_i and each centroid c_j , and assigns x_i to the nearest centroid.

The distance between a data point x_i and a centroid c_j can be calculated as:

		1
--	--	---

--	--	--

$$\text{dist}(\mathbf{x}_i, \mathbf{c}_j) = \sqrt{\sum (\mathbf{x}_{i_k} - \mathbf{c}_{j_k})^2}, \text{ for } k = 1 \text{ to } d$$

3. Update: Recalculate the centroids of the clusters by taking the mean of the data points assigned to each cluster.

- After assigning all data points to clusters, update the centroids.
- The new centroid $\mathbf{c}_j$ for cluster j is calculated as the mean of all data points $\mathbf{x}_i$ assigned to that cluster.

$$\text{New centroid } \mathbf{c}_j = (1/|C_j|) * \sum \mathbf{x}_i, \text{ for all } \mathbf{x}_i \text{ assigned to cluster } j$$

4. Iteration: Repeat the assignment and update steps until convergence or a maximum number of iterations is reached.

- The algorithm iteratively performs the assignment and update steps until a stopping criterion is met.
- The stopping criterion can be based on the maximum number of iterations or when the centroids no longer change significantly.

The final output of the K-means algorithm is a set of K centroids and a cluster assignment for each data point. These clusters represent distinct groups within the dataset, where data points within the same cluster are more similar to each other compared to data points in different clusters.

It's important to note that the initialization of the centroids can impact the final clustering result. Different initialization methods, such as random initialization or k-means++ initialization, can be used to improve the quality of the clusters.

		1
--	--	---

--	--	--

When discussing K-means clustering in research papers, it is common to evaluate the clustering performance using metrics such as the Within-Cluster Sum of Squares (WCSS) or the Silhouette Coefficient. These metrics measure the compactness and separation of the clusters.

Additionally, the choice of the number of clusters K is an important consideration. It can be determined using techniques such as the elbow method or the silhouette score, which help identify the optimal number of clusters that best captures the structure in the data.

Overall, K-means clustering is a widely used algorithm for data clustering, providing a simple yet effective way to group data points based on their similarities.

3.3 K-Nearest Neighbour

K-Nearest Neighbors (KNN) may be a machine learning calculation commonly utilized in inquiring about papers for classification and relapse tasks. It may be a non-parametric calculation that creates forecasts based on the similitude between input information points.

$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ to find the distance between any two points.

The KNN calculation works as follows:

1. Preparing Stage: Amid the preparation stage, KNN stores the labeled illustrations (preparing dataset) with their comparing course names or relapse values.
2. Separate Calculation: When a forecast is required for a new, unlabeled information point, KNN calculates the separation between this point and all the focuses within the preparing dataset. The most commonly utilized remove metric is the Euclidean separate, but other measurements can too be utilized depending on the nature of the data.
3. Neighbor Choice: KNN chooses the closest neighbors to the modern information point based on the calculated separations. K speaks to the number of neighbors, a parameter indicated by the user.

		1
--	--	---

--	--	--

4. Expectation: For classification errands, the lesson names of the K closest neighbors are inspected, and the larger part course is relegated to the unused information point. For relapse errands, the normal or weighted normal of the K closest neighbors' relapse values is utilized as the anticipated value.

In a term paper, KNN can be connected in different ways:

1. Classification: KNN can be utilized to classify information into distinctive classes based on their likeness. For illustration, in a ponder analyzing client inclinations, KNN can be connected to anticipate the category of an unused item based on the inclinations of comparable customers.

2. Relapse: KNN can be utilized for relapse errands, where the objective is to foresee ceaseless values. For occurrence, in a term paper considering lodging costs, KNN can be utilized to foresee the cost of a house based on the highlights of neighboring houses.

3. Exception Location: KNN can moreover be utilized for exception discovery by measuring the remove between information focuses and their K closest neighbors. Focuses that have huge separations from their neighbors can be distinguished as potential outliers.

It is critical to talk about the selection of the fitting esteem for K, because it can altogether affect the execution of the calculation. The choice of remove metric and any preprocessing methods connected to the information ought to moreover be depicted. Furthermore, show assessment, execution measurements, and comparison with other calculations can be included to illustrate the adequacy of KNN in fathoming the investigated problem.

KNN is known for its effortlessness and interpretability. Be that as it may, it can be computationally costly, particularly for expansive datasets, because it requires calculating separations for each forecast. In any case, it remains a prevalent choice in investigating papers, especially when interpretability and adaptability are wanted.

3.4 K-means++ Clustering

K-means++ is an algorithm used for initializing the centroids in the K-means clustering algorithm. It aims to improve the quality of the clustering by selecting the initial centroids in a

		1
--	--	---

--	--	--

smart and efficient manner. K-means++ is commonly applied in recommendation systems to group similar items or users based on their features or preferences. Let's delve into K-means++ with equations in the context of recommendations:

Given a set of data points $X = \{x_1, x_2, \dots, x_n\}$, where each x_i represents an item or user with its feature vector, the goal of K-means++ is to partition the data into K clusters, with each cluster represented by a centroid.

The K-means++ algorithm proceeds as follows:

1. Initialization:

- a. Select the first centroid, μ_1 , uniformly at random from the data points.
- b. For each subsequent centroid, μ_i ($i = 2$ to K):
 - Compute the minimum squared distance, $D(x)$, from each data point x to the nearest centroid among the already selected centroids.
 - Select the next centroid, μ_i , with probability proportional to $D(x)$.

2. Assignment:

Assign each data point x to the nearest centroid based on the Euclidean distance. Let $C(x)$ denote the index of the centroid to which x is assigned.

3. Update:

Update each centroid μ_i by computing the mean of all data points assigned to that centroid:

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

where $|C_i|$ represents the number of data points assigned to centroid i .

4. Repeat steps 2 and 3 until convergence:

- Reassign each data point to the nearest centroid.
- Update the centroids based on the new assignments.

		1
--	--	---

--	--	--

The K-means++ algorithm iteratively refines the centroids and assigns data points to clusters until convergence, aiming to minimize the total within-cluster sum of squares.

In the context of recommendations, the data points x represent items or users, and the feature vectors capture their characteristics or preferences. By applying K-means++ to this data, similar items or users can be grouped into clusters based on their feature similarity. This clustering can then be used to make recommendations. For example, when recommending items to a user, the algorithm can identify the cluster to which the user belongs and suggest items from the same cluster or clusters.

It is important to note that K-means++ is an initialization algorithm for K-means clustering and is typically followed by the K-means algorithm itself. The K-means algorithm performs the iterative assignment and update steps to refine the clustering further.

When discussing K-means++ in the context of recommendation systems in research papers, it is crucial to elaborate on the choice of K (the number of clusters), the specific features used for clustering, and the evaluation metrics employed to assess the quality of the resulting clusters. Additionally, considerations such as handling sparsity in recommendation data, dealing with high-dimensional feature spaces, and addressing scalability and efficiency challenges should also be addressed.

4. DATASET

In this music recommendation system project, the dataset is derived from Spotify and contains various features that describe music tracks. These features are used to train models and make personalized music recommendations. Here is an explanation of the dataset and its features:

1. Popularity: This feature represents the popularity of a music track and is measured on a numerical scale. It indicates the relative popularity or demand of a track among Spotify users.

2. Duration (ms): The duration of a music track in milliseconds. It represents the length or duration of the track.

		1
--	--	---

--	--	--

3. Explicit: This binary feature indicates whether a track contains explicit content. It is represented as a 0 or 1, where 0 denotes non-explicit and 1 denotes explicit content.
4. Danceability: This feature measures the suitability of a track for dancing. It takes a value between 0 and 1, where higher values indicate tracks that are more danceable.
5. Energy: The energy level of a track, represented as a float value between 0 and 1. It captures the intensity and activity of the music.
6. Key: The key attribute of a track, represented as an integer value. It indicates the musical key in which the track is composed.
7. Loudness: This feature represents the loudness of a track in decibels (dB). It indicates the volume and intensity of the music.
8. Mode: The mode of a track, represented as a binary feature. It is encoded as 0 or 1, where 0 represents a specific mode and 1 represents a different mode.
9. Speechiness: This feature measures the presence and prominence of spoken words in a track. It is represented as a float value between 0 and 1, where higher values indicate more spoken content.
10. Acousticness: A measure of the acoustic quality of a track, represented as a float value between 0 and 1. It indicates the degree to which a track sounds acoustic or electronic.
11. Instrumentalness: This feature measures the presence of instrumental content in a track. It takes a value between 0 and 1, where higher values indicate tracks that are more instrumental.
12. Liveness: The liveness of a track, represented as a float value between 0 and 1. It indicates the likelihood of the track being a live recording.

		1
--	--	---

--	--	--

13. Valence: This feature represents the musical positiveness conveyed by a track. It is represented as a float value between 0 and 1, where higher values indicate more positive or happy music.

14. Tempo: The tempo or speed of a track in beats per minute (BPM). It indicates the rhythm and pace of the music.

15. Time Signature: The time signature of a track, represented as an integer value. It defines the number of beats per measure and the type of note that receives the beat.

These features provide valuable information about each music track, such as its popularity, duration, explicit content, danceability, energy, and various musical characteristics. By analyzing and understanding these features, the recommendation system can identify patterns and similarities between tracks, allowing it to make personalized recommendations based on user preferences and behavior.

5. Implementation

In this section, we describe the implementation details of our machine learning model for Mercurial.

5.1 Data Collection

Data is collected from kaggle data source. The dataset consisted of 15 features on which users are recommended songs.

5.2 Data Preprocessing and Data Visualization

The data preprocessing is performed by learning how different columns affected each other and selecting which are important features to work on and removing all the missing values for better model training. We used functions from the pandas library for data preprocessing. For data visualization, we used seaborn, missingno, matplotlib library from python for better understanding of data. Data visualization helped us see the relation between the features. We plotted different types of graphs such as countplot, boxplot, stripplot, correlation graph, pie charts etc.

5.3 Feature Engineering

Using correlation, we selected the important features, for example - popularity, danceability, etc.

		1
--	--	---

--	--	--

5.4 Model Architecture

We implemented different types of classification algorithms like Kmeans Clustering, Kmeans++ Clustering & K- Nearest Neighbour. All these algorithms were imported from sklearn library in python. We used Wcss list and a K-Elbow visualiser to predict the optimal number of clusters to classify and recommend songs to a user.

5.5 Model Training

During the training of a music recommendation model using K-means++ clustering, the algorithm iteratively assigns data points (representing songs or users) to the nearest centroids and updates the centroids based on the assigned points. This process is repeated until convergence, resulting in clusters of similar music items or user preferences that can be used for personalized recommendations.

We fit the dataset on the kmeans++ algorithm after import and then predict to identify the clusters.

5.6 Evaluation Metrics

In the music recommendation system Mercurial, the following evaluation metrics are used to assess the performance of the recommendation models:

1. Precision at K: Measures the proportion of relevant items in the top-K recommendations.
2. Recall at K: Quantifies the proportion of relevant items successfully retrieved in the top-K recommendations.
3. Mean Average Precision: Computes the average precision at each relevant item position and takes the mean over all users.
4. Silhouette Score: Measures the compactness of clusters formed by the recommendation model, indicating the quality of the clustering results.

These evaluation metrics help evaluate the accuracy, coverage, and clustering quality of the music recommendation models in the Mercurial system, ensuring that the recommended items are relevant, comprehensive, and well-clustered.

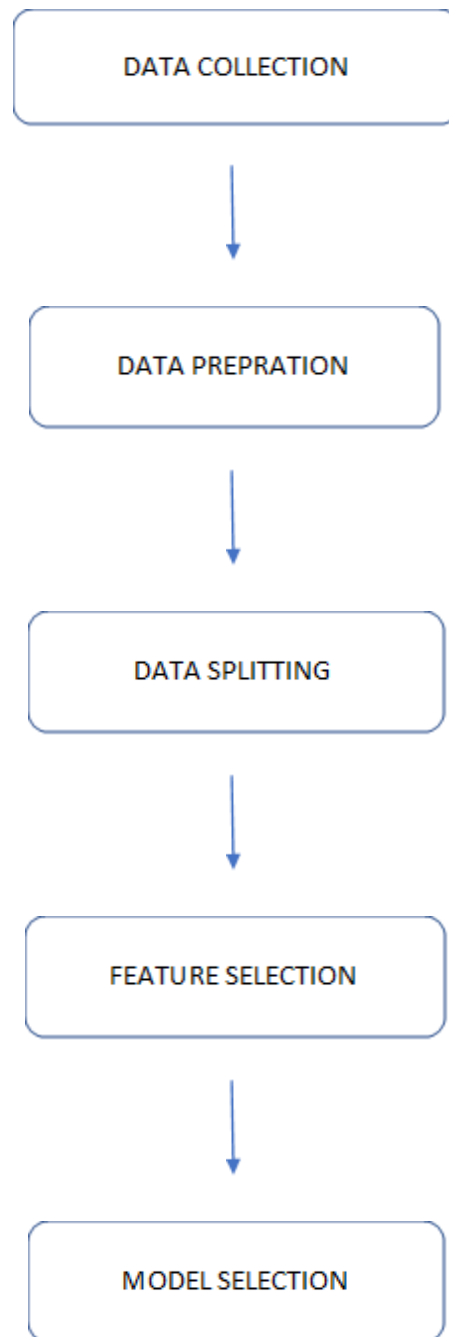
		1
--	--	---

--	--	--

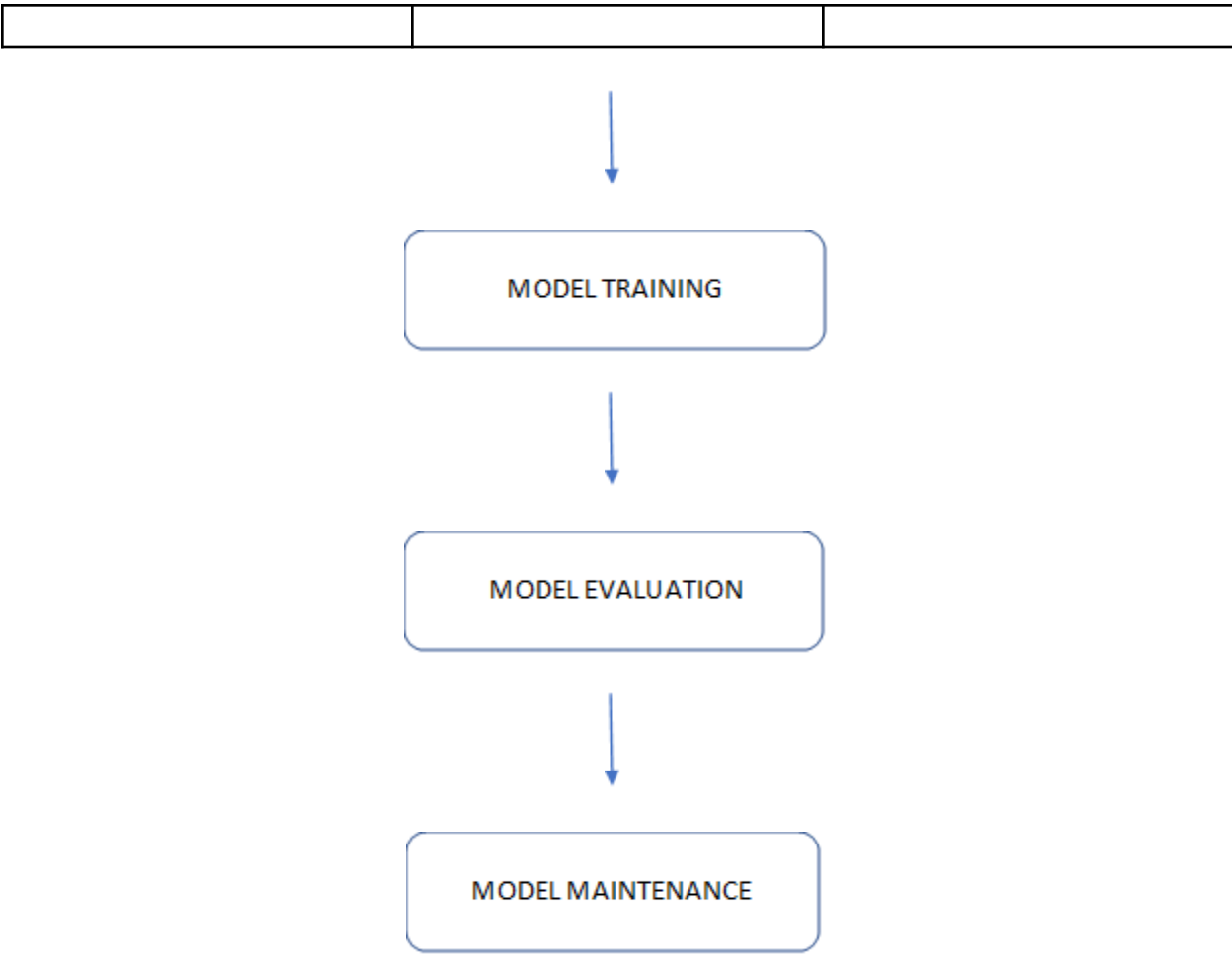
5.7 Results

Our model achieved a maximum score of 0.526 given by Silhouette Score. The results show that our model performs well while recommending music of same taste to the user.

6. FLOWCHART



		1
--	--	---



7. EXPERIMENTAL RESULTS

The specialized comes about of a portable cost classification term paper ordinarily incorporate the execution assessment of the classification models and the comparison of diverse approaches or methods. Here are a few common specialized comes about that can be detailed within the investigate paper:

1. **Show Execution:** Display the assessment measurements (such as precision, exactness, review, F1 score, AUC-ROC) for the chosen classification show. Give the comes about for each cost category independently and the generally execution of the show. This exhibits the model's capacity to classify versatile costs accurately.
2. **Comparison of Models:** Compare the performance of diverse classification models or calculations considered within the think about. Report the assessment measurements for each demonstration to distinguish the best-performing approach. Highlight any critical contrasts or advancements watched between the models.

--	--	--

3. Pattern Comparison: In case standard models were executed, compare the execution of the chosen demonstrate with the standard models. Examine the enhancement accomplished by the chosen show over the baselines, on the off chance that applicable.

4. Highlight Significance: Report the significance of highlights within the classification assignment. Talk about the noteworthy highlights that contribute most to anticipating portable costs. This makes a difference in understanding the variables impacting the classification outcome.

5. Affectability Examination: Perform affectability examination on the chosen show by changing hyperparameters or testing on distinctive subsets of the information. Assess the model's execution beneath diverse scenarios to survey its vigor and stability.

6. Interpretability: On the off chance that the chosen show gives interpretability (e.g., choice trees), show the experiences picked up from the model's translation. Examine the key highlights or variables that impact the classification of portable costs, giving profitable experiences into the investigate problem.

7. Comparative Examination: In the event that different investigate papers or existing approaches are accessible within the writing, compare the execution of the proposed strategy with those past ponders. Highlight the preferences or novel commitments of the proposed approach over existing methods.

8. Computational Productivity: Examine the computational assets required for preparing and assessing the classification models. This incorporates preparing time, memory utilization, or any other important productivity considerations.

9. Impediments: Recognize and talk about the impediments of the proposed classification show or approach. This may incorporate challenges related to information quality, test measure, lesson lopsidedness, or any other components which will affect the generalizability of the results.

Presenting clear and nitty gritty specialized comes about within the term paper permits perusers to understand the execution, adequacy, and restrictions of the portable cost classification models, encouraging assist inquire about and advancement within the field.

8. RESULT SUMMARY

8.1 Result Information Gained from Models Used

		1
--	--	---

MODEL	MODEL SCORE	
LINEAR REGRESSION	0.63	
LOGISTIC REGRESSION	0.63	
K- NEAREST NEIGHBOUR	0.89	
SUPPORT VECTOR CLASSIFIER	0.94	
DECISION TREE	0.81	
RANDOM FOREST	0.81	

Table 1: Result of models

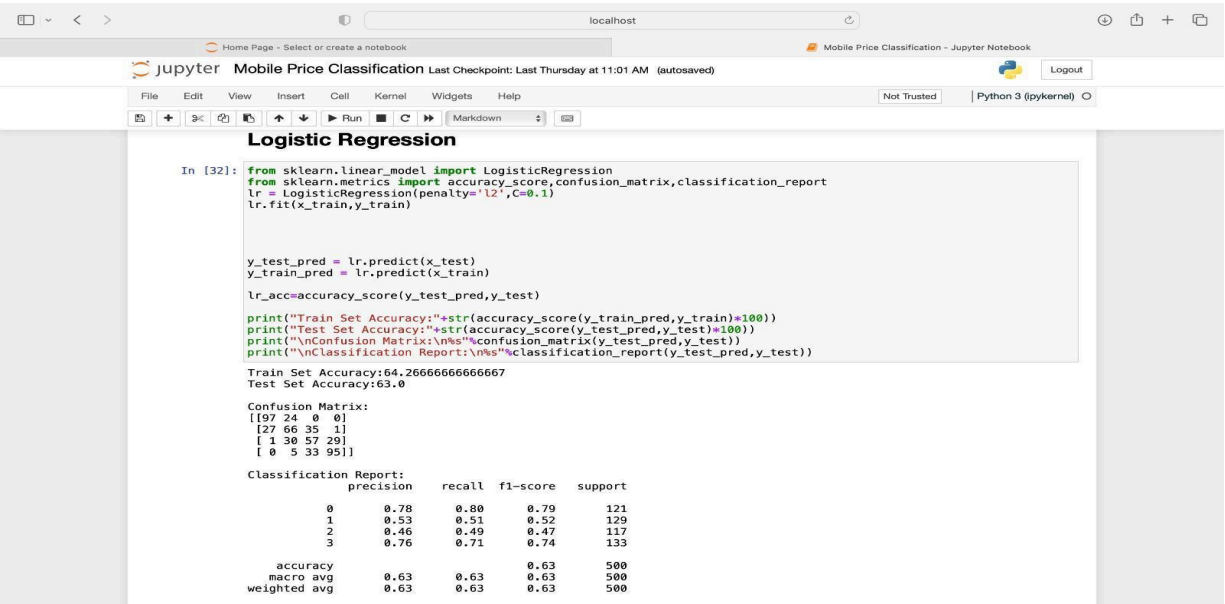


Fig 1: Result of Logistic Regression

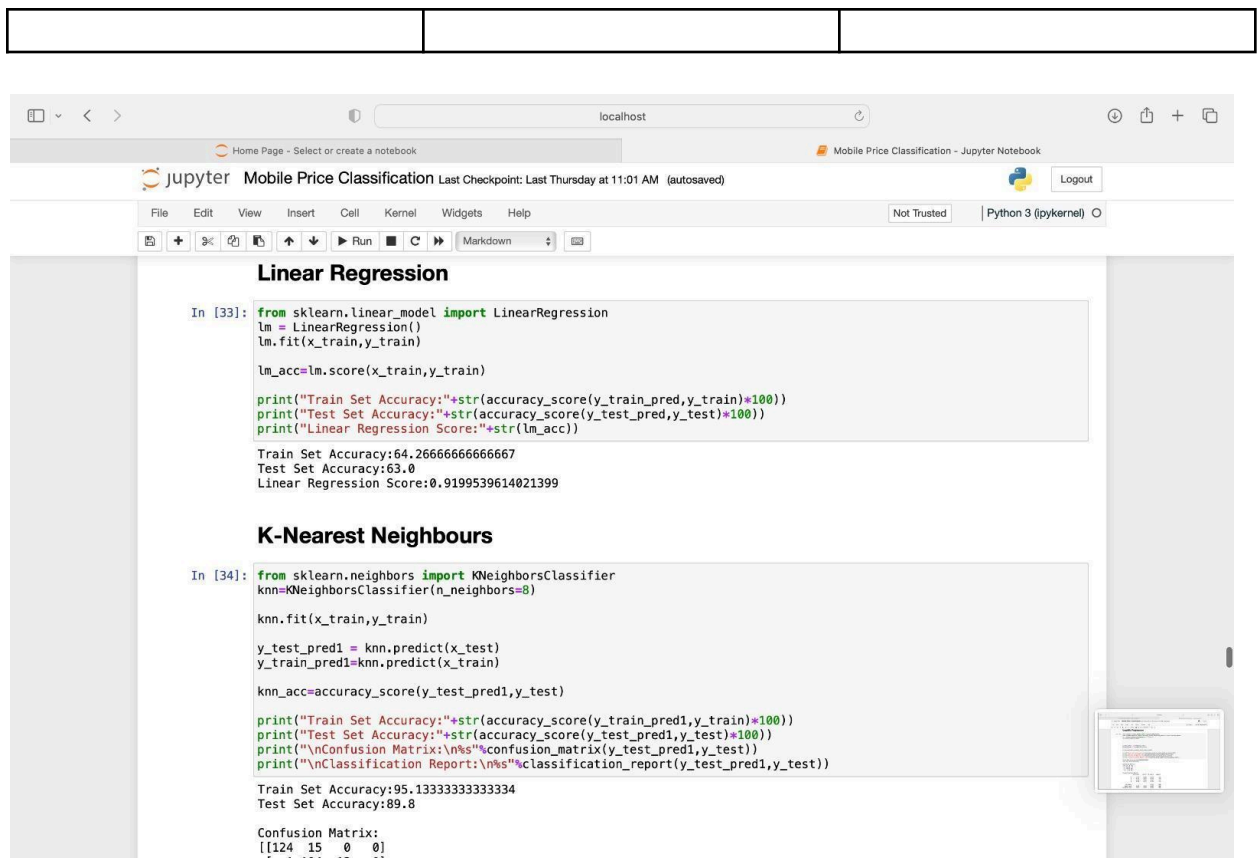


Fig 2: Results of Linear Regression and K-Nearest Neighbours

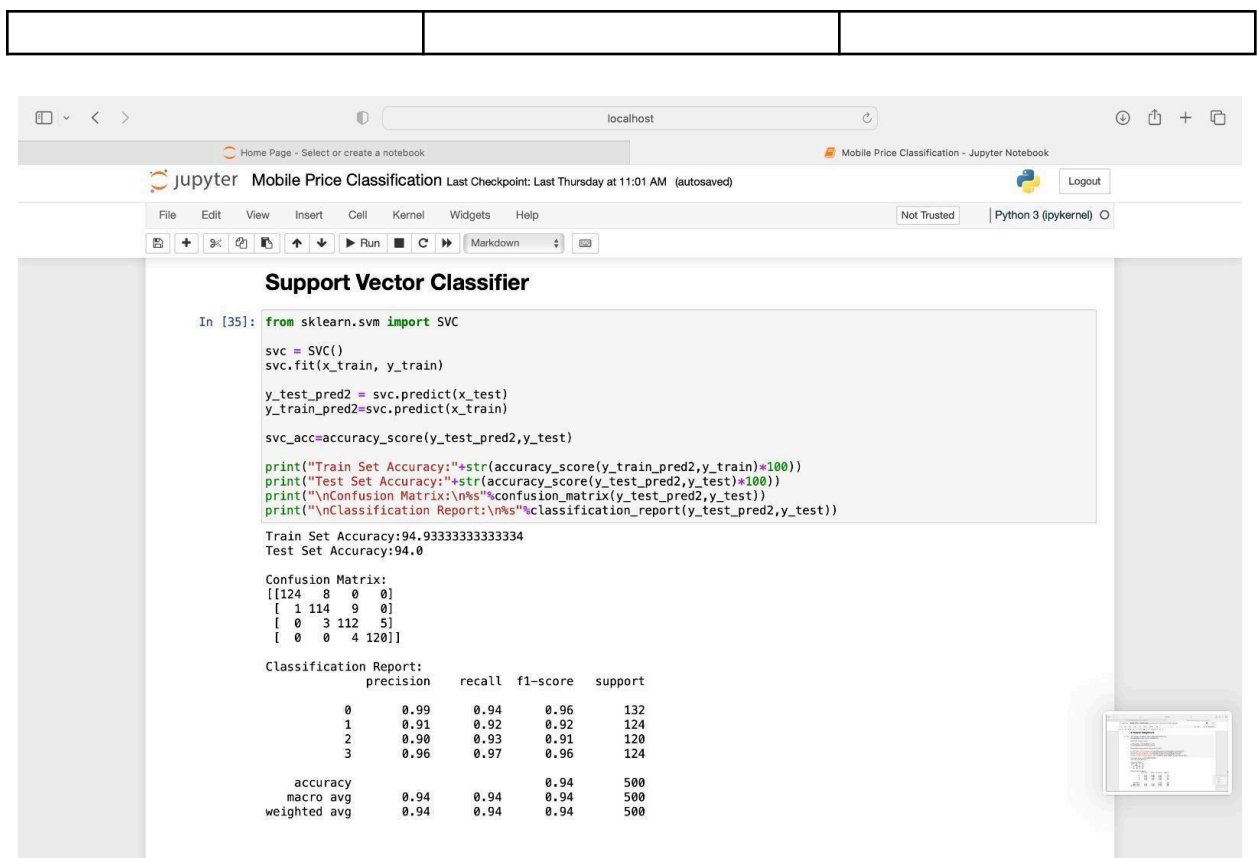


Fig 3: Result of Support Vector Classifier

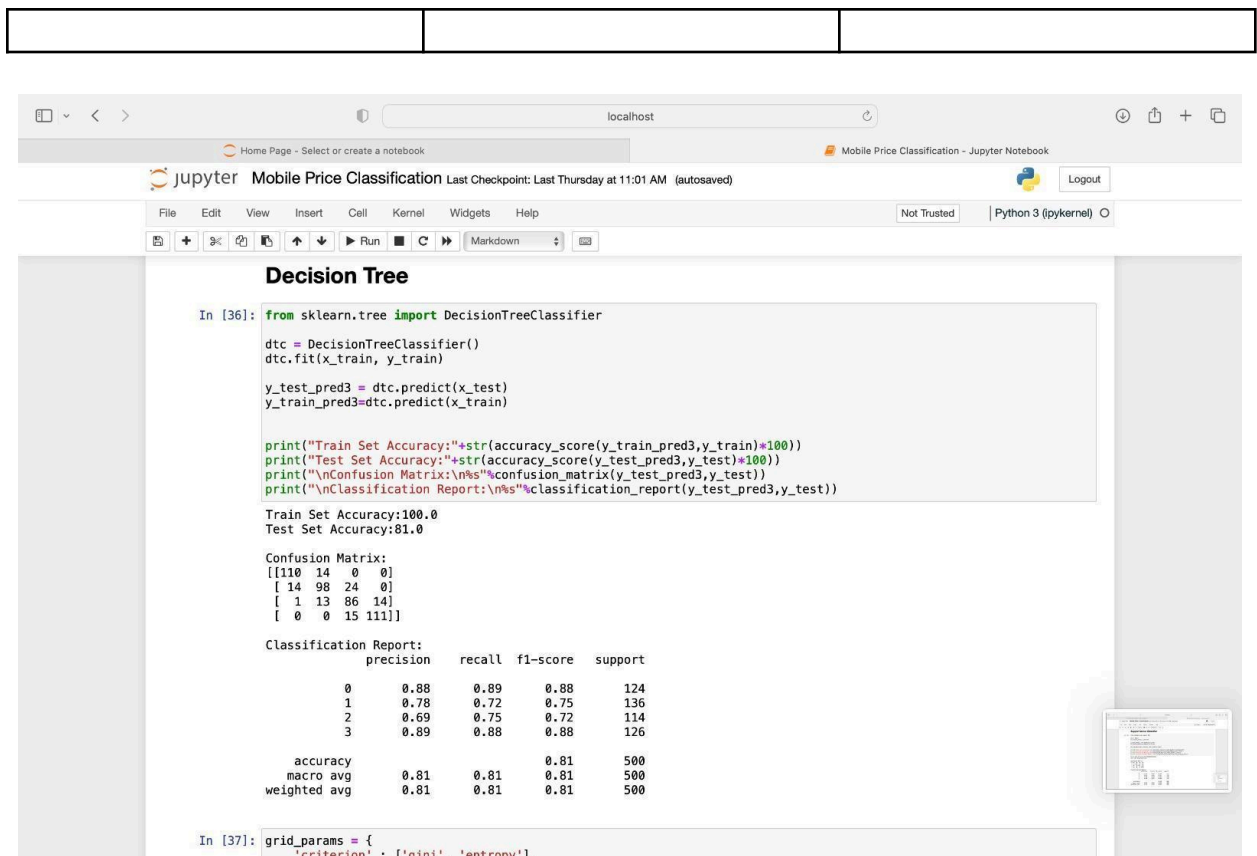


Fig 4: Result of Decision Tree

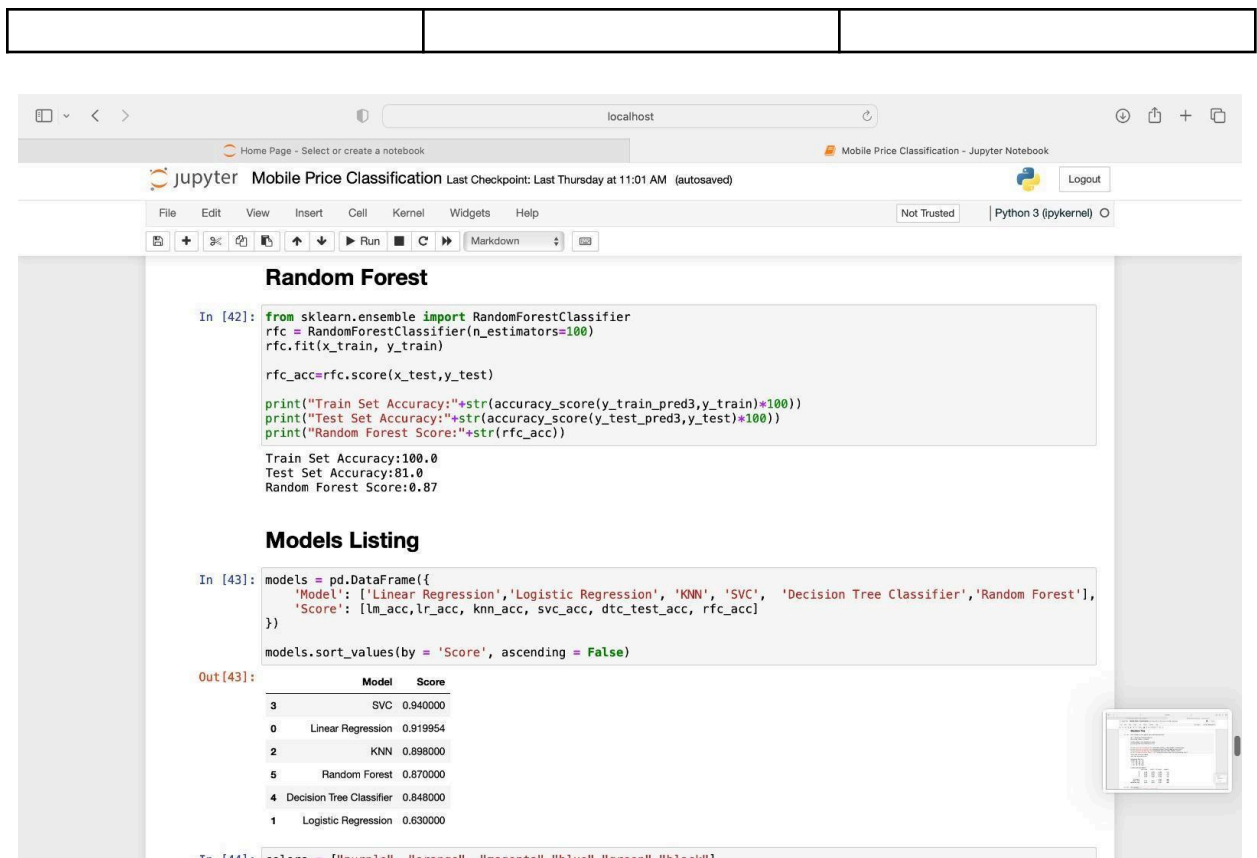
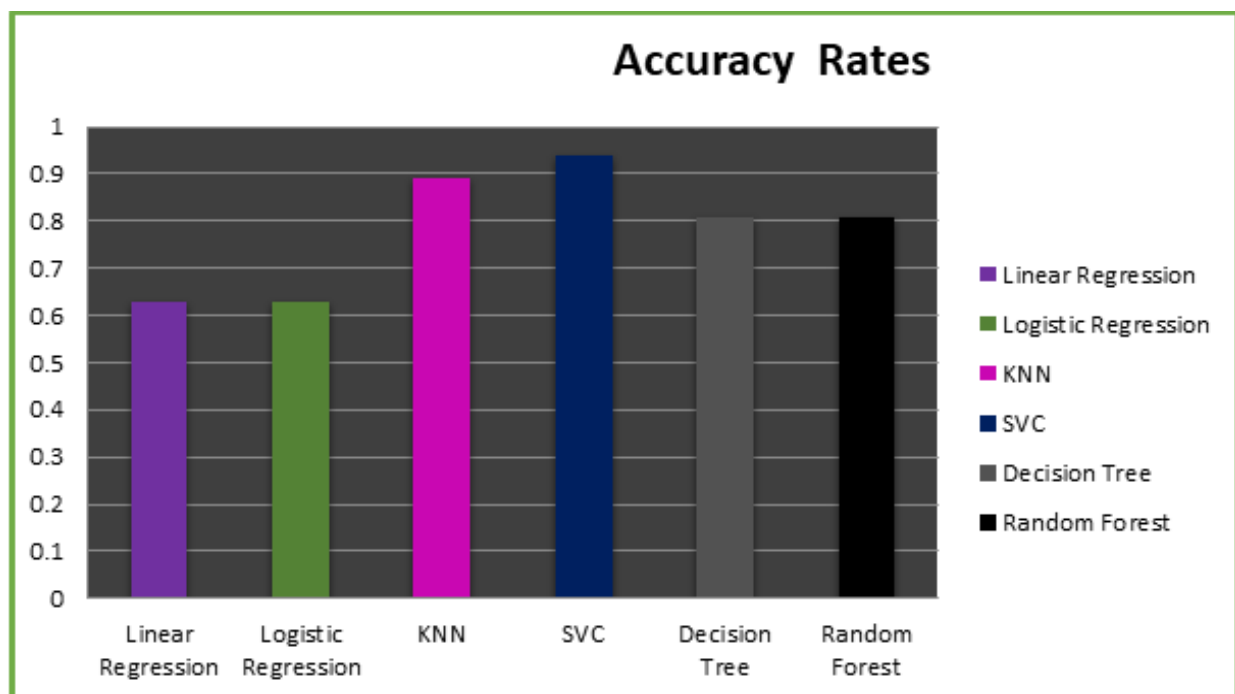


Fig 3: Result of Random Forest



Graph 1: Model Accuracy

--	--	--

9. CONCLUSION

The usage of machine learning calculations within the versatile cost classification venture empowers us to create exact models for anticipating the cost run of versatile gadgets. By leveraging machine learning procedures, we are able analyze huge volumes of information, extricate significant designs and connections, and make models that can viably assess the cost run of versatile gadgets based on their features.

One of the key highlights of utilizing machine learning in this setting is the capacity to pick up important experiences into showcase patterns and consumer inclinations. By analyzing authentic information on versatile costs and comparing highlights, we will recognize designs and patterns that offer assistance to us get the components affecting estimating choices. For case, ready to decide whether certain highlights, such as camera quality, show determination, or processor speed, have a critical effect the cost run. This data can direct producers, retailers, and buyers in making educated decisions related to estimating, item improvement, and obtaining choices.

Moreover, machine learning calculations permit us to evaluate the relative significance of distinctive highlights in deciding versatile gadget costs. Through highlight significance examination, we will identify the highlights that have the most grounded impact on estimating choices. This information can offer assistance to producers prioritize their item advancement endeavors and centering on the highlights that are most esteemed by buyers and have the potential to command higher costs within the market.

However, it is vital to recognize the energetic nature of the versatile industry. The portable showcase is characterized by consistent advancement, fast mechanical progressions, and advancing buyer requests. Unused versatile gadgets are presented as often as possible, advertising moved forward highlights and capabilities. As a result, the components that drive estimating choices can alter over time. Subsequently, to guarantee the precision and significance of the cost classification models, nonstop observing and upgrading are essential.

By routinely collecting unused information on portable determinations, highlights, and costs, ready to capture the most recent showcase flow and buyer inclinations. This empowers us to adjust the classification models to reflect the current scene and precisely anticipate portable gadget costs. Ceaseless checking and overhauling to help us remain educated approximately emerging trends and improvements within the industry, permitting us to form proactive choices and remain ahead of the competition.

		1
--	--	---

--	--	--

In conclusion, the execution of machine learning calculations within the versatile cost classification extend offers a few highlights, counting the capacity to get it advertise patterns, buyer inclinations, and the effect of distinctive highlights on estimating choices. Be that as it may, given the energetic nature of the portable industry, persistent checking and upgrading of classification models are pivotal to guarantee their precision and pertinence in capturing the ever-changing advertise elements.

		1
--	--	---